EVA Analysis Application

By: Nandakishore Vinayakrishnan, SID 23070854

1. Summary

I inherited a single 122-line Python script that plots cumulative EVA (spacewalk) time and counts EVAs by duration. I worked on that code rather than replacing it. I assessed it, split it by responsibility into a small `eva/` package, added a country comparison (`--compare`), and covered it with 55 automated tests. Two architectural decision records (ADRs) explain the main choices.

2. Activity 1: Assessment of the inherited code

`evacodev2.py` reads `eva-data.json` (375 EVAs, 330 usable once entries without a date, duration or country are dropped). It asks for a country and a year range, filters, builds a running total of hours, counts EVAs as short (under 4 h), standard (4 to under 7 h) or long (7 h+), saves and shows a plot, and prints the counts and percentages. The git history shows five rounds of work; each added to the same file.

Input	Observed result
USA, 1965 to 1975	Works: 29 short, 9 standards, 7 long
No filters (all data)	Works: 71 short, 187 standards, 72 long
Unknown country (“Narnia”)	Quietly treated as all countries
No matches, reversed years, non-numeric year	Crashes (`ZeroDivisionError` or `ValueError`)

Responsibility	Location
Reading data	Module level, lines 5 to 6
Validating and preparing	Module level, lines 8 to 27
Calculating	`aggregate_data`, `evaluate_duration_occurances`, `calculate_occurance_percentages`
Presenting	`plot_graph` and the final `print` calls
Coordinating and input	`get_clean_inputs` and module-level lines 118 to 122

- Everything runs at import time and there is no `main()`. `aggregate_data` converts typed text, filters, totals and counts in one function. `get_clean_inputs` silently turns an unknown country into "all countries". `calculate_occurance_percentages` returns formatted strings, and `plot_graph` draws, saves and blocks.
- Functions read the shared globals, the `records` and the `countries`. `aggregate_data` assumes `records` are `(date, hours, country)` tuples built elsewhere, indexes them by position and expects years as strings, so the analysis depends on both the loader and the prompt. The file name, the `"all countries"` string and the 4 h / 7 h thresholds are hard-coded.
- Importing the file reads data, prompts and plots. `aggregate_data`, `get_clean_inputs` and `plot_graph` depend on globals, `input()` or the display. Only `evaluate_duration_occurances` is a pure function.

3. Activity 2: Proposed architecture

The script becomes the `eva/` package, one module per responsibility. `evacodev2.py` stays as a thin entry point, so `python evacodev2.py` works as before.

Module	Responsibility
`app.py`	Coordinates a run: loads data, picks the mode, calls the others in order. No logic of its own
`data_source.py`	Reads the JSON file; knows the format and nothing else
`preparation.py`	Drops incomplete entries, converts dates and durations, sorts
`models.py`	`EvaRecord` (date, duration in hours, country), shared by preparation and analysis
`analysis.py`	Pure calculations and rules: filtering, running totals, duration categories, percentages, country selection, year-range check
`comparison.py`	Compares two countries, built on `analysis`

Module	Responsibility
`presentation.py`	The plot and all text output
`user_input.py`	Prompts, the `--compare` flag, parsing typed text
`errors.py`	`InvalidQueryError` for requests that cannot be answered

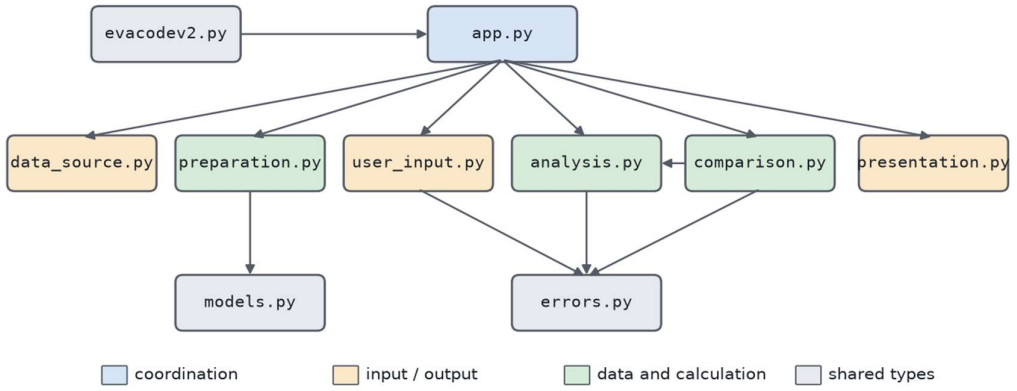


Figure 1. Module dependencies (an arrow means "imports"). app.py also imports errors.py to catch it.

Modules communicate through plain functions and one small data class. The main interfaces are `load_raw_records(path)`, `prepare_records(raw)`, `filter_records(records, country, start_year, end_year)`, `cumulative_duration(records)`, `count_duration_categories(records)`, `compare_countries(records, a, b, start, end)`, `plot_cumulative_duration(...)` and `format_comparison(result)`. "All countries" is `None` rather than a magic string, and percentages stay numbers until `presentation` formats them.

- These boundaries follow what can change independently, since a new file format touches only `data_source`, a new cleaning rule only `preparation`, a new statistic only `analysis`, a new output only `presentation`, etc.
- Every function in a module serves it's one job: `preparation` answers "is this entry usable, and what is its typed form?", `analysis` turns records into numbers, `presentation` turns results into something a person sees. `EvaRecord` has its own module, so `preparation` and `analysis` do not depend on each other.
- `aggregate_data` reads a global list, indexed it by position and expected year strings from `input()`. Now `filter_records` takes a list of `EvaRecord` and integer years, so the loader, prompt and analysis can each change alone, and tests can call it with handwritten records.

ADR-001: Split the single script into modules by responsibility

The app used to be one script where reading, cleaning, analysis, output and control flow were tangled. Importing it ran the whole program, the main functions read global variables, so they could not be run on test data, and one function took raw typed text and did the filtering, totalling and counting. A single file is not allowed, and the new feature needed to reuse the existing calculations.

Possible Alternatives.

1. Keep one file and tidy it (add `main()`, remove globals). Cheap, but this doesn't meet the spec.
2. Split by the existing function list (`functions.py`, `helpers.py`). Several files and arbitrarily assigned boundaries.
3. Split by responsibility into one flat package. **Chosen.**
4. Classes, interfaces and dependency injection. Decouples well but adds machinery a program this size would not be justifiable.

Hence, we create the `eva/` package with a module per responsibility, keep `evacodev2.py` as the entry point, use plain functions and one dataclass, no module-level state, and let only `app` depend on everything else.

Each module changes for one reason. Passing data as arguments removes the global-state coupling. Pure functions can be tested with a few handwritten records. Functions and one dataclass are the smallest thing that works. The comparison can sit on top of `analysis` without touching loading or preparation. Changes to the format, cleaning rules, statistics, output or prompts each stay in one module, and calculations can be unit tested. The costs are more files to follow, a shared dependency on the shape of `EvaRecord`, an `app.py` that must stay free of logic, and no tests at the start, so "behaviour unchanged" had to be shown by comparing outputs.

4. Activities 3 and 4: Restructuring and the new feature

The logic was moved, instead of being rewritten as per the project spec. For example, `aggregate_data` became `filter_records`, `cumulative_duration` and `count_duration_categories`, and `plot_graph` became `plot_cumulative_duration`. Before changing anything, I recorded the original's output on 12 inputs (output, exit status and a hash of the saved plot) to compare afterwards.

`I also added a new feature, python evacodev2.py –compare, which` asks for Country A, Country B, a start year and an end year (both inclusive) and reports, for each country, the total EVAs, total hours and average hours, then which country has more total time or a tie:

```
EVA comparison, 1990 to 2000 (inclusive)
USA:
  Total EVAs: 36
  Total duration (hours): 233.13
  Average duration (hours): 6.48
Russia:
  Total EVAs: 58
  Total duration (hours): 272.45
  Average duration (hours): 4.70
Result: Russia has the greater total EVA duration
```

- `comparison.py` builds on `filter_records` and `cumulative_duration`, so nothing is calculated twice. Loading, preparation and the calculation modules were not changed for the feature. Only `app`, `presentation` and `user_input` gained code.
- A country with no matching EVAs shows 0 EVAs, 0 hours and an average of "unavailable". Equal totals are reported as a tie, using `math.isclose` because `0.1 + 0.2 != 0.3` in floating point.
- Reversed years, non-numeric years and unknown countries print a message instead of crashing (ADR-002). Because the brief says the application must handle these, the original mode got the same handling.
- `--compare` keeps the original prompts and output exactly as they were.

ADR-002: Report bad requests with one exception, handled in app

The comparison must cope with reversed years, non-numeric years and unknown countries without crashing. `user_input` sees the typed text and `comparison` knows the rules, and either needs a way to say "this request is invalid" without printing, exiting or depending on the other. The inherited code just crashed with a raw `ValueError` or `ZeroDivisionError`.

Possible Alternatives.

1. Re-prompt inside `user_input` until valid. Friendly, but it would need the data and the rules and could not be tested without a fake console.
2. Return `None` or an error value. Every caller must remember to check, making future devs lives harder.
3. Let `ValueError` bubble up and catch it in `app`. Simple but catching errors that broadly could also hide real bugs.
4. A dedicated `InvalidQueryError` in its own small module. **Chosen.**

`eva/errors.py` defines `InvalidQueryError(ValueError)`. `analysis`, `comparison` and `user_input` raise it. Only `app` catches it, prints “`Cannot compare: <reason>`” or “`Cannot analyse: <reason>`” and returns normally.

`This way, comparison` stays a pure function, and invalid input is tested with `assertRaises`, with no console. The lower layers describe the problem, and `app` decides how to tell the user. A narrow exception type does not swallow unrelated bugs, and `user_input` and `comparison` need not import each other.

Every invalid-input path in both modes has a test and ends in a readable message, and both modes share one year-range check. The costs are one more tiny module, a changed (improved) behaviour for inputs that used to crash, and an exit status of 0 after an error message, which a later change could make non-zero.

5. Activity 5: Verification and evaluation

`python -m unittest -v` runs 55 tests, all passing. Each uses known inputs and checks exact results.

Test file	Covers
`test_comparison.py`	Hand-worked counts, totals and averages; both end years included; no matches; ties (including float rounding); reversed years; unknown countries

Test file	Covers
`test_analysis.py`	Filtering, running totals, percentages, the 4 h / 7 h category boundaries, country selection, year-range check
`test_preparation.py`	Incomplete entries dropped, durations converted, records sorted
`test_user_input_and_presentation.py`	Both sets of prompts (blank and non-numeric years), the `--compare` flag, text output
`test_app.py`	Both modes end to end with scripted input; bad input is reported and nothing is plotted when there is nothing to plot
`test_real_data.py`	The real data file gives the numbers the original script gave (29/9/7 and 71/187/72), and the comparison matches an independent calculation

Q. Would the tests catch a mistake?

A. I broke the code four ways on a scratch copy (an exclusive end year, `>=` for the year check, `0` instead of "unavailable" for an empty average, a shifted category boundary). Each made tests fail.

Q. Does the old behaviour still work?

A. Of the 12 inputs recorded from the original script, 8 give identical output and plots. The other 4 are the inputs that used to crash (reversed years, a non-numeric year, two empty results). They now print a message, exit normally and draw no empty plot.

Measure	Before	After
Source files	1	11
Lines (no blanks or comments)	98	377
Functions	5	24
Longest function	37 lines	32 lines
Statements run on import	10	0
Functions reading shared globals	2	0
Automated tests	0	55
Features	1	2

Q. What modules were changed?

A. Five files: two new (`comparison.py`, `errors.py`) and three extended (`app`, `presentation`, `user_input`). The old script would have meant editing the one file that holds everything.

Q. How do your changes affect dependencies and testing?

A. Figure 1 shows the dependencies, with no cycles. `analysis` and `comparison` take plain records, so they are tested without files or a display. Before, importing the script already read a file and prompted for input.

Q. Are responsibilities easier to identify?

A. Yes. Each of the five is now a named module.

Q. Are there any instances of code duplication?

A. The comparison reuses existing functions, the `"all countries"` string became `None`, and both modes share one year check. Some filter-then-total repetition remains.

Q. Is there any unnecessary complexity in the program?

A. Some. It is 11 files instead of 1 and nearly four times the lines (much of it docstrings and the new feature), and calls now span files. There are no frameworks or new dependencies. A two or three-module split would also have worked, but five responsibilities keep each module short and each boundary easy to explain.

Known limitations: An unknown country in the original mode still means "all countries", as it always did, whereas the comparison rejects it. EVAs recorded as `0:00` (cancelled ones) still count as EVAs with zero hours. Country names are case-sensitive. A missing data file stops the program with a normal Python error.

6. Activity 6: Hand-off

Another developer needs Python 3.10 or newer and the packages in `requirements.txt` (matplotlib). From the repository root:

```
pip install -r requirements.txt
python evacodev2.py             # cumulative duration and categories
python evacodev2.py --compare   # compare two countries
python -m unittest -v           # run the tests
```

`README.md` has set-up and the module map, `docs/` and `docs/adr/` hold the written records, and `tests/` holds the tests.